

오프라인

2026.06.23 TUE

파이썬 프로그래밍 1일차 강의자료

파이썬 첫 만남 · 변수와 자료형 · 입력과 출력 · 연산자 · 조건문

오늘의 한 줄 목표

- 내 컴퓨터에 파이썬을 깔고, 변수와 if문으로 작동하는 작은 프로그램을 직접 만든다.

박재현 / viver12@hoseo.edu

일정



오늘의 시간표

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

교시	주제	이론	실습
1	파이썬 첫 만남 & 개발 환경	프로그래밍·컴퓨팅 사고·파이썬	설치 · Hello World
2	변수와 자료형	변수·자료형·형변환	변수 가지고 놀기
3	입력과 출력	print·input·f-string	단위 변환기 · BMI
4	연산자	산술·비교·논리	계산기 모음
5	조건문 if	if/elif/else·들여쓰기	학점·윤년·가위바위보

오프라인 운영 리듬

- 각 60분: 이론 20분 + 실습 30분 + 휴식 10분
- 설치부터 조건문까지 하루 안에 한 번 연결
- 완성 프로그램을 매 교시 하나씩 남깁니다

TAKEAWAY

오늘의 목표: 내 컴퓨터에서 파이썬을 실행하고, 변수와 if문으로 작은 프로그램을 직접 만든다.

오리엔테이션

환영합니다

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

외우지 않기

- 비전공자 기준으로 예시를 먼저 보고, 손으로 만들어 보며 익힙니다.

에러는 힌트

- 에러 메시지는 혼내는 말이 아니라 다음 행동을 알려주는 안내문입니다.

작은 완성

- BMI, 학점, 가위바위보처럼 작동하는 프로그램을 남깁니다.

오늘 끝나면 만들 수 있는 것

여러 간단한 프로그램

TAKEAWAY

처음엔 따라치고, 곧 스스로 만듭니다.

1교시

1교시 — 파이썬 첫 만남 & 개발환경

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 프로그래밍이 무엇인지 감을 잡고, 파이썬을 설치해 첫 코드를 실행한다.

이론 20분

실습 30분

휴식 10분

이론

프로그래밍이란 무엇인가?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 프로그래밍 = 컴퓨터에게 일을 시키는 것

예시

- "프로그램"은 컴퓨터에게 주는 명령서(레시피)

주의

- 우리가 매일 쓰는 것들이 전부 프로그램:

연결

- 카카오톡, 유튜브, 배달앱, 게임, 인스타그램...

실행하며 확인할 질문

- 오늘부터 우리는 쓰는 사람에서 만드는 사람이 됩니다

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

프로그래밍 = 컴퓨터에게 일을 시키는 것

이론

컴퓨터는 어떻게 생각할까?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

01

시킨 것만 한다 (알아서 안 함)

02

순서대로 한다 (위에서 아래로)

03

정확히 한다 (애매하면 에러)

순서가 바뀌면 결과도 바뀝니다

- 컴퓨터는 똑똑해 보이지만 사실 고지식합니다
- 컴퓨터의 3가지 특징:
- 사람: "대충 알아서 해줘" / 컴퓨터: "정확히 단계별로 알려줘"

TAKEAWAY

프로그래밍은 문제를 작게 나누고, 실행 순서를 정확히 적는 일입니다.

이론

컴퓨터 vs 사람 — 커피 타기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

01

컵을 꺼낸다

02

커피를 2스푼 넣는다

03

뜨거운 물 150ml를 붓는다

04

10초간 젓는다

순서가 바뀌면 결과도 바뀝니다

- 사람에게: "커피 좀 타줘" → 알아서 함
- 컴퓨터에게는 이렇게 잘게 쪼개야 함:
- 한 단계라도 빠지면? → 이상한 커피가 나옵니다

TAKEAWAY

프로그래밍은 문제를 작게 나누고, 실행 순서를 정확히 적는 일입니다.

이론

컴퓨팅적 사고력이란?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- Computational Thinking = 문제를 컴퓨터가 풀 수 있게 바꿔 생각하는 힘

예시

- 4가지 도구로 이루어집니다:

주의

- 분해 (Decomposition)
- 패턴 찾기 (Pattern)

연결

- 추상화 (Abstraction)
- 알고리즘 (Algorithm)

실행하며 확인할 질문

- 이건 코딩뿐 아니라 모든 문제 해결에 쓰이는 사고법

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

Computational Thinking = 문제를 컴퓨터가 풀 수 있게 바꿔 생각하는 힘

이론

컴퓨팅적 사고 ① 분해

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 큰 문제를 작은 문제로 쪼개기

예시

- 예: "여행 계획 세우기"

주의

- 교통편 정하기 / 숙소 예약 / 일정 짜기 / 예산 계산

연결

- 큰 덩어리는 막막하지만, 잘게 나누면 하나씩 해결 가능

실행하며 확인할 질문

- 프로그래밍도 똑같습니다: 큰 프로그램 = 작은 기능들의 합

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

큰 문제를 작은 문제로 쪼개기

이론

컴퓨팅적 사고 ② 패턴 찾기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 반복되는 규칙을 발견하기

예시

- 예: 구구단 2단, 3단, 4단... → "
어떤 수 × 1~9" 라는 공통 패턴

주의

- 예: 출석부 1번, 2번, 3번... → "
번호가 1씩 증가"

연결

- 패턴을 찾으면 → 반복문으로
한 번에 처리 (내일 배웁니다!)

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

반복되는 규칙을 발견하기

이론

컴퓨팅적 사고 ③ 추상화

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 중요한 것만 남기고 불필요한 건 숨기기

예시

- 예: 지하철 노선도 → 실제 거리
.건물은 무시, 역과 연결만 표시

주의

- 예: 자동차 운전 → 엔진 원리
몰라도 핸들·페달만 알면 됨

연결

- 프로그래밍: 복잡한 기능을 이름 하나로 묶어 사용 (예: print)

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

중요한 것만 남기고 불필요한 건 숨기기

이론

컴퓨팅적 사고 ④ 알고리즘

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

01

물 550ml를 냄비에 붓는다

02

물이 끓을 때까지 기다린다

03

면과 스프를 넣는다

04

4분 30초 기다린다

05

불을 끈다

순서가 바뀌면 결과도 바뀝니다

- 문제를 푸는 단계별 순서
- 예: "라면 끓이기 알고리즘"
- 코드 = 알고리즘을 파이썬 언어로 옮긴 것

TAKEAWAY

프로그래밍은 문제를 작게 나누고, 실행 순서를 정확히 적는 일입니다.

이론

순서가 중요하다!

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 알고리즘은 순서가 바뀌면 결과 달라집니다

예시

- 라면 예시: "불을 끈다 → 면을 넣는다" 면? 설익은 라면!

주의

- 예: "양말 신기 → 신발 신기" (O) vs "신발 → 양말" (X)

연결

- 컴퓨터는 위에서 아래로 순서대로 실행 → 순서 설계가 핵심

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

알고리즘은 순서가 바뀌면 결과가 달라집니다

이론

프로그래밍 언어란?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 사람의 말(한국어)과 컴퓨터의 말(0과 1)은 다릅니다

예시

- 프로그래밍 언어 = 사람과 컴퓨터의 중간 통역

주의

- 종류가 많습니다: 파이썬, 자바, C, 자바스크립트...

연결

- 각각 장단점이 있지만, 우리는 가장 쉬운 파이썬으로 시작!

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

사람의 말(한국어)과 컴퓨터의 말(0과 1)은 다릅니다

이론

파이썬의 탄생

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 1991년, 네덜란드 개발자 귀도 반 로섬이 만듦

예시

- 이름의 유래: 좋아하던 코미디 쇼 "몬티 파이썬"

주의

- 뱀이 아닙니다! (로고가 뱀이라 오해 多)

연결

- 처음부터 "읽기 쉽고 배우기 쉽게"를 목표로 설계

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

1991년, 네덜란드 개발자 귀도 반 로섬이 만듦

이론

파이썬이 쉬운 이유

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 다른 언어 (자바)로 "Hello" 출력:
- 파이썬으로 같은 일:
- 한 줄이면 끝! → 비전공자에게 최고의 첫 언어

Python example

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}  
  
print("Hello")
```

TAKEAWAY

다른 언어 (자바)로 "Hello" 출력:

이론

파이썬은 어디에 쓰일까? (1)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 웹 개발: 인스타그램, 유튜브, 넷플릭스 백엔드

예시

- 데이터 분석: 엑셀로 버거운 수십만 줄 데이터 처리

주의

- 인공지능(AI): ChatGPT, 이미지 생성 AI도 파이썬 기반

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

웹 개발: 인스타그램, 유튜브, 넷플릭스 백엔드

이론

파이썬은 어디에 쓰일까? (2)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 업무 자동화: 반복되는 엑셀 정리, 파일 이름 바꾸기, 이메일 발송

예시

- 과학·연구, 금융·주식 분석, 게임, 로봇

주의

- 비전공자에게 가장 실용적인 건 "업무 자동화"

연결

- 매일 1시간 걸리던 일을 1초로!

실행하며 확인할 질문

- 우리 과정의 최종 목표도 여기까지 갑니다

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

업무 자동화: 반복되는 엑셀 정리, 파일 이름 바꾸기, 이메일 발송

이론

코드는 어떻게 실행될까?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 우리가 쓴 코드 → 컴퓨터가 알아듣는 말로 번역되어야 함

예시

- 번역 방식 2가지:

주의

- 컴파일러: 코드 전체를 한꺼번에 번역 후 실행 (C, 자바)

연결

- 인터프리터: 코드를 한 줄씩 번역하며 바로 실행 (파이썬)

실행하며 확인할 질문

- 이 개념이 언제 필요한가?
- 잘못 쓰면 어떤 에러가 나는가?
- 다음 실습에서 어디에 연결되는가?

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

우리가 쓴 코드 → 컴퓨터가 알아듣는 말로 번역되어야 함

이론

인터프리터 vs 컴파일러

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

	컴파일러	인터프리터(파이썬)
번역 단위	전체 한꺼번에	한 줄씩
실행 속도	빠름	상대적으로 느림
결과 확인	번역 끝나야	바로바로
입문자	어려움	쉬움

읽는 법

- 파이썬은 한 줄씩 바로 실행 → 결과를 즉시 보며 배우기 좋음

TAKEAWAY

파이썬은 한 줄씩 바로 실행 → 결과를 즉시 보며 배우기 좋음

이론

개발 환경 — 무엇이 필요할까?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 요리하려면 재료와 주방이 필요하듯, 코딩에도 도구가 필요

예시

- 두 가지를 설치합니다:

주의

- 비유: 엔진(파이썬) + 운전석(VS Code)

연결

- Python 3.12+ → 코드를 실행하는 엔진

실행하며 확인할 질문

- VS Code → 코드를 편하게 쓰는 에디터(작업 공간)

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

요리하려면 재료와 주방이 필요하듯, 코딩에도 도구가 필요

실습

파이썬 설치

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

따라 하기

- Windows
- macOS
- python.org → Downloads → Python 3.12 다운로드·실행
- 반드시 체크: Add python.exe to PATH
- 설치가 안될 시 '관리자 권한'으로 실행

변형 미션

- Install Now → 완료
- cmd에서 python --version → Python 3.12.x
- python.org → macOS 설치 파일 다운로드·실행
- 터미널에서 python3 --version 확인 (Mac은 python3, pip3)

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

VS Code & 첫 코드

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
>>> print("Hello, World!")
Hello, World!
>>> 1 + 1
2
```

실습 포인트

- 첫 프로그래밍 성공!
- code.visualstudio.com → 설치 → 실행
- 확장(Extensions) 설치: Python(MS), Korean Language Pack(선택)
- 터미널에서 python(Mac은 python3) → >>> 나오면:

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

나를 소개하는 파일

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
print("이름: _____")  
print("학과: _____")  
print("파이썬으로 만들고 싶은 것: _____")
```

실습 포인트

- hello.py 파일 만들기
- 아래를 채워 출력:
- 실행: 오른쪽 위 ▶ 버튼
- 추가 미션: 좋아하는 음식, MBTI, 한 줄 각오도 출력!

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

휴식

휴식 10분

잠깐 멈추고, 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 변수와 자료형 — "값을 담는 상자"를 배웁니다

눈 쉬기

질문 정리

파일 저장

다음 실습 준비

TAKEAWAY

쉬는 시간에도 작성한 파일은 저장해 둡니다.

2교시

2교시 — 변수와 자료형

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 변수에 값을 담고, 4가지 자료형과 형 변환을 구분한다.

이론

변수(Variable)란?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 변수 = 값을 담아두는 상자
- 상자에 이름표를 붙이고 값을 넣습니다
- name 상자에 "홍길동", age 상자에 24 가 들어감
- 꺼내 쓸 땐 이름표(변수명)를 부릅니다:

Python example

```
name = "홍길동"  
age = 24  
  
print(name) # 홍길동
```

TAKEAWAY

변수 = 값을 담아두는 상자

이론

= 의 진짜 의미

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 수학에서 = 는 "양쪽이 같다"
- 프로그래밍에서 = 는 "오른쪽 값을 왼쪽 상자에 넣어라(저장)"
- 그래서 이런 것도 가능합니다:

Python example

```
x = 10    # 10을 x에 넣어라  
  
x = 10  
x = 20    # 이제 x에는 20 (덮어쓰기)  
print(x)  # 20
```

TAKEAWAY

수학에서 = 는 "양쪽이 같다"

이론

변수에 값 다시 넣기 (재할당)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 오른쪽 money - 3000 을 먼저 계산(7000) → 다시 money 에 저장
- 변수는 언제든지 새 값으로 바꿀 수 있다

Python example

```
money = 10000
print(money)    # 10000

money = money - 3000 # 3000원 썼다
print(money)    # 7000
```

TAKEAWAY

오른쪽 money - 3000 을 먼저 계산(7000) → 다시 money 에 저장

이론

변수끼리 계산하기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 변수에 담긴 값들로 새로운 값을 계산해 또 다른 변수에 저장
- 이게 프로그래밍의 기본 흐름입니다

Python example

```
korean = 90
english = 85
math = 95

total = korean + english + math
average = total / 3

print(total) # 270
print(average) # 90.0
```

TAKEAWAY

변수에 담긴 값들로 새로운 값을 계산해 또 다른 변수에 저장

이론

변수를 왜 쓸까?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 재사용 수정 편함 의미 전달 (코드가 읽힘)

Python example

```
# 변수 없이 (이름 바뀌면 다 고쳐야 함)
print("홍길동님 환영합니다")
print("홍길동님 포인트는 1000점")

# 변수로 (한 곳만 고치면 끝)
name = "홍길동"
print(name, "님 환영합니다")
print(name, "님 포인트는 1000점")
```

TAKEAWAY

재사용 수정 편함 의미 전달 (코드가 읽힘)

이론

변수 이름 규칙 ① 가능한 것

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 영문자, 숫자, 밑줄(_) 사용
- 대소문자 구분: Age 와 age 는 다른 변수
- 길어도 됨 — 오히려 의미 있게:
- 관례: 단어 사이를 _ 로 연결 (snake_case)

Python example

```
user_name = "김파이"  
total_price = 15000  
is_student = True
```

TAKEAWAY

영문자, 숫자, 밑줄(_) 사용

이론

변수 이름 규칙 ② 안 되는 것

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 숫자로 시작: 1age (X) → age1 (O)
- 공백 포함: user name (X) → user_name (O)
- 특수문자: price\$, total! (X)
- 예약어: if, for, print, class 등은 이미 쓰임

Python example

```
1st = 10 # SyntaxError
my age = 20 # SyntaxError
```

TAKEAWAY

숫자로 시작: 1age (X) → age1 (O)

이론

좋은 변수명 vs 나쁜 변수명

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 코드는 컴퓨터보다 사람이 더 많이 읽습니다

Python example

```
# 나쁜 예 (무슨 값인지 모름)
a = 50000
b = 0.1
c = a - a * b

# 좋은 예 (읽으면 이해됨)
price = 50000
discount_rate = 0.1
final_price = price - price * discount_rate
```

TAKEAWAY

코드는 컴퓨터보다 사람이 더 많이 읽습니다

이론

자료형(Data Type)이란?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 값에도 종류(타입)가 있습니다

예시

- 마치 마트의 물건이 식품/의류/가전으로 나뉘듯

주의

- 파이썬 기본 자료형 4가지:

연결

- 정수 int, 실수 float, 문자열 str, 불리언 bool

실행하며 확인할 질문

- 타입에 따라 할 수 있는 일이 다릅니다 (숫자는 계산, 글자는 연결)

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

값에도 종류(타입)가 있습니다

이론

정수(int)와 실수(float)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 정수 int: 소수점 없는 수 → 10, 0, -3, 2025
- 실수 float: 소수점 있는 수 → 3.14, -0.5, 36.5
- 나눗셈 / 결과는 항상 실수: $10 / 2 \rightarrow 5.0$

Python example

```
age = 24      # int
height = 175.5 # float
temperature = -2 # int
```

TAKEAWAY

정수 int: 소수점 없는 수 → 10, 0, -3, 2025

문자열(str)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 문자열 = 글자(텍스트), 반드시 따옴표로 감쌉니다
- 작은따옴표 ' ' 와 큰따옴표 " " 둘 다 OK
- 따옴표 없으면? → 변수명으로 착각해서 에러
- 숫자도 따옴표로 감싸면 글자: "123" 은 문자열!

Python example

```
name = "홍길동"  
city = '서울'  
message = "파이썬은 재밌다!"
```

TAKEAWAY

문자열 = 글자(텍스트), 반드시 따옴표로 감쌉니다

이론

불리언(bool)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 불리언 = 참(True) 또는 거짓(False) 단 두 값
- 첫 글자 대문자 주의: True, False
- 어디에 쓰나? → 조건 판단 (5교시 조건문의 핵심!)
- 예: "성인인가?" → True / False 로 답

Python example

```
is_adult = True  
is_raining = False
```

TAKEAWAY

불리언 = 참(True) 또는 거짓(False) 단 두 값

이론

type() — 타입 확인하기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- type() 은 값의 자료형을 알려주는 도구
- 헛갈릴 때 직접 찍어 확인하는 습관!

Python example

```
print(type(24))    # <class 'int'>
print(type(3.14))  # <class 'float'>
print(type("안녕")) # <class 'str'>
print(type(True))  # <class 'bool'>
```

TAKEAWAY

type() 은 값의 자료형을 알려주는 도구

이론

24 와 "24" 는 다르다!

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 겉보기엔 같아 보여도 완전히 다른 값
- 숫자 24 는 더하기 가능, 글자 "24" 는 이어붙이기만

Python example

```
print(24)      # 숫자 24
print("24")    # 글자 24
print(type(24)) # int  → 계산 가능
print(type("24")) # str → 그냥 글자
```

TAKEAWAY

겉보기엔 같아 보여도 완전히 다른 값

이론

숫자 + vs 글자 +

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 같은 + 기호가 타입에 따라 다르게 동작!
- 그래서 타입을 아는 게 중요합니다

Python example

```
print(3 + 5)      # 8   (숫자 → 덧셈)
print("3" + "5")  # 35  (글자 → 이어붙이기)
print("안녕" + "하세요") # 안녕하세요
print("하하" * 3)  # 하하하하하하 (글자 × 숫자 = 반복)
```

TAKEAWAY

같은 + 기호가 타입에 따라 다르게 동작!

이론

형 변환(Type Casting)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 타입을 바꾸는 것 → 글자 ↔ 숫자
- 활용:

Python example

```
int("24")    # 24 (글자 → 정수)
float("3.14") # 3.14 (글자 → 실수)
str(100)     # "100" (숫자 → 글자)

num = "10"
num = int(num) # 이제 숫자 10
print(num + 5) # 15
```

TAKEAWAY

타입을 바꾸는 것 → 글자 ↔ 숫자

형 변환 주의점

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- int() 는 소수점을 버림(내림 아님, 그냥 잘라냄)
- 변환 불가능한 값은 에러 → 입력값 확인 필요

Python example

```
int("안녕")    # 에러! 숫자가 아닌 글자
int(3.9)        # 3 (반올림 x, 소수점 버림!)
round(3.9)      # 4 (반올림은 round)
int("3.5")      # 에러 (소수 글자는 int 불가)
int(float("3.5")) # 3 (float 거쳐서)
```

TAKEAWAY

int() 는 소수점을 버림(내림 아님, 그냥 잘라냄)

실습

숫자 vs 글자 체험

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
print(3 + 5)          # 8
print("3" + "5")      # 35
print("안녕" + "하세요") # 안녕하세요
print("=" * 20)        # =====
print(int("3") + int("5")) # 8
```

실습 포인트

- 여러 경우를 직접 출력하며 차이를 느껴봅시다:

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

변수.타입 확인

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
name = "김파이"
age = 20
height = 168.5
is_freshman = True

print("이름:", name, "/ 타입:", type(name))
print("나이:", age, "/ 타입:", type(age))
print("키:", height, "/ 타입:", type(height))
print("신입생?", is_freshman, "/ 타입:", type(is_freshman))
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 모든 경우를 확인합니다.

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

형 변환 미니 퀴즈

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
print(10 + 20)    # ?
print("10" + "20") # ?
print(int("10") + 20) # ?
print(str(10) + "20") # ?
print(int(3.9))   # ?
print("10" * 3)   # ?
```

실습 포인트

- 각 줄의 결과를 예측한 뒤 실행해 확인:
- 정답: 30 / "1020" / 30 / "1020" / 3 / "101010"

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

휴식

휴식 10분

잠깐 멈추고, 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 입력과 출력 — 프로그램이 사용자와 대화를 시작합니다!

눈 쉬기

질문 정리

파일 저장

다음 실습 준비

TAKEAWAY

쉬는 시간에도 작성한 파일은 저장해 둡니다.

3교시

3교시 — 입력과 출력

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: input()으로 입력받고 f-string으로 깔끔히 출력해 실용 계산기를 만든다.

이론 20분

실습 30분

휴식 10분

이론

print() 다시 보기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- print() = 괄호 안의 내용을 화면에 보여주는 명령
- 가장 많이 쓰는 도구, 결과 확인의 기본

Python example

```
print("Hello") # 글자 출력
print(100)     # 숫자 출력
print(3 + 5)   # 계산 결과 출력 → 8
```

TAKEAWAY

print() = 괄호 안의 내용을 화면에 보여주는 명령

이론

print() 여러 값 출력

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 콤마(,)로 여러 개를 한 줄에 출력
- 콤마 사이에는 자동으로 공백 한 칸이 들어감

Python example

```
name = "홍길동"
age = 24
print(name, age)          # 홍길동 24
print("이름:", name, "나이:", age) # 이름: 홍길동 나이: 24
```

TAKEAWAY

콤마(,)로 여러 개를 한 줄에 출력

이론

print() 옵션 (sep, end)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- sep = 값 사이에 넣을 구분자 (기본: 공백)
- end = 출력 끝에 넣을 것 (기본: 줄바꿈)

Python example

```
print("2025", "06", "23", sep="-") # 2025-06-23
print("안녕", end=" ")
print("하세요") # 안녕 하세요
```

TAKEAWAY

sep = 값 사이에 넣을 구분자 (기본: 공백)

이론

input() — 사용자에게 묻기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- input() = 키보드 입력을 받아서 변수에 저장
- 괄호 안 글자는 안내 문구(질문)
- 실행:

Python example

```
name = input("이름을 입력하세요: ")  
print("안녕하세요," , name, "님!")
```

```
이름을 입력하세요: 홍길동  
안녕하세요, 홍길동 님!
```

TAKEAWAY

input() = 키보드 입력을 받아서 변수에 저장

이론

input()의 동작 원리

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 프로그램이 멈추고 사용자 입력을 기다림
- 사용자가 타이핑 후 Enter
- 입력한 값이 변수에 저장되고 다음 줄로

Python example

```
age = input("나이: ") # 여기서 멈춰 기다림  
print("입력 완료!") # Enter 후 실행
```

TAKEAWAY

프로그램이 멈추고 사용자 입력을 기다림

이론

input()의 함정

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- input()의 결과는 항상 문자열(str)!
- 화면엔 20 으로 보여도 컴퓨터에겐 글자 "20"
- 숫자로 계산하려면 → 형 변환 필수

Python example

```
age = input("나이: ") # 사용자가 20 입력
print(age + 1)        # 에러! "20" + 1 불가
```

TAKEAWAY

input()의 결과는 항상 문자열(str)!

이론

input() + 형 변환

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 한 줄로 합치는 게 일반적:

Python example

```
age = input("나이: ") # "20"  
age = int(age)        # 숫자 20으로 변환  
print(age + 1)        # 21
```

```
age = int(input("나이: ")) # 정수로  
height = float(input("키: ")) # 실수로
```

TAKEAWAY

한 줄로 합치는 게 일반적:

이론

어떤 형으로 변환할까?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

입력 종류	변환 함수	예
나이, 개수, 점수	int()	int(input())
키, 몸무게, 돈(소수)	float()	float(input())
이름, 주소 (글자)	변환 x	input() 그대로

읽는 법

- 계산할 숫자면 변환, 글자면 그냥 사용

TAKEAWAY

계산할 숫자면 변환, 글자면 그냥 사용

이론

문자열 따옴표 종류

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 큰·작은 따옴표 기능은 같음
- 따옴표 안에 따옴표 넣을 땐 서로 다른 종류 사용

Python example

```
a = "큰따옴표"
b = '작은따옴표'
c = "그는 '안녕'이라고 말했다" # 안에 다른 따옴표 OK
```

TAKEAWAY

큰·작은 따옴표 기능은 같음

이론

f-string — 깔끔한 출력 ①

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 문자열 앞에 f를 붙이고 {변수}로 값을 끼워넣기
- 콤마 방식보다 훨씬 읽기 쉽고 깔끔!

Python example

```
name = "홍길동"  
age = 24  
print(f"{name}님은 {age}살입니다.")  
# 홍길동님은 24살입니다.
```

TAKEAWAY

문자열 앞에 f를 붙이고 {변수}로 값을 끼워넣기

이론

f-string ② 계산식도 OK

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- {} 안에는 변수뿐 아니라 계산식도 가능
- 코드가 짧고 의도가 분명해집니다

Python example

```
age = 24
print(f"내 년엔 {age + 1}살이 됩니다.") # 내 년엔 25살
price = 5000
print(f"3개 가격: {price * 3}원")      # 3개 가격: 15000원
```

TAKEAWAY

{ } 안에는 변수뿐 아니라 계산식도 가능

이론

f-string ③ 소수점 포매팅

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- {값:.1f} → 소수점 1자리까지 (반올림)
- 계산 결과가 지저분할 때 필수

Python example

```
pi = 3.141592
print(f'{pi:.2f}') # 3.14 (소수점 2자리)
print(f'{pi:.0f}') # 3 (반올림, 정수처럼)

bmi = 22.857
print(f'BMI: {bmi:.1f}') # BMI: 22.9
```

TAKEAWAY

{값:.1f} → 소수점 1자리까지 (반올림)

이론

f-string ④ 천 단위 콤마 & 정렬

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- :, 큰 금액 읽기 쉽게, :> :< 표·영수증 줄맞춤

Python example

```
price = 1200000
print(f'{price:,}') # 1,200,000 (천 단위 콤마)
print(f'{price:,}원') # 1,200,000원

name = "홍길동"
print(f'[{name:>10}]') # [   홍길동] (오른쪽 정렬)
print(f'[{name:<10}]') # [홍길동   ] (왼쪽 정렬)
```

TAKEAWAY

:, 큰 금액 읽기 쉽게, :> :< 표·영수증 줄맞춤

이론

옛날 방식 vs f-string

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 결과는 비슷하지만 f-string이 읽기 쉽고 실수 적음
- 앞으로 우리는 주로 f-string 사용

Python example

```
name = "홍길동"; age = 24

# 콤마 방식 (공백 신경, 따옴표 많음)
print(name, "님은", age, "살입니다.")

# f-string 방식 (추천!)
print(f"{name}님은 {age}살입니다.")
```

TAKEAWAY

결과는 비슷하지만 f-string이 읽기 쉽고 실수 적음

실습

단위 변환기 모음

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
# ① cm → inch
cm = float(input("cm: "))
print(f"{cm}cm = {cm / 2.54:.2f}inch")

# ② 섭씨 → 화씨
c = float(input("섭씨: "))
print(f"{c}°C = {c * 9/5 + 32:.1f}°F")

# ③ 달러 → 원화 (1달러=1350원)
usd = float(input("달러: "))
print(f"${usd} = {usd * 1350:,.0f}원")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 모든 경우를 확인합니다.

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

BMI 계산기

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
print("=== BMI 계산기 ===")
weight = float(input("몸무게(kg): "))
height = float(input("키(cm): "))

height_m = height / 100
bmi = weight / (height_m * height_m)
print(f"당신의 BMI는 {bmi:.1f} 입니다.")
```

실습 포인트

- {bmi:.1f} → 소수점 1자리

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

나만의 변환기 (택1 이상)

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

따라 하기

- 현실 소재 중 골라 만들어 보세요:
- ⌚ 초 → 시·분·초 (//, % 활용)
- 걸음 수 → 칼로리: 걸음 $\times$ 0.04 kcal

2. 실행 결과 확인

3. 값을 바꿔 보기

변형 미션

- 키(cm) → 적정 몸무게: $(키 - 100) \times 0.9$
- 부가세 계산기: 가격 → 공급가 + 부가세(10%)

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

휴식

휴식 10분

잠깐 멈추고, 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 연산자 — 계산과 비교의 모든 것!

눈 쉬기

질문 정리

파일 저장

다음 실습 준비

TAKEAWAY

쉬는 시간에도 작성한 파일은 저장해 둡니다.

4교시

4교시 — 연산자

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: 산술·비교·논리 연산자를 이해하고, 다양한 계산 프로그램을 만든다.

이론 20분

실습 30분

휴식 10분

이론

연산자란?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

01

산술 연산자 (계산)

02

비교 연산자 (크기 비교)

03

논리 연산자 (조건 합치기)

순서가 바뀌면 결과도 바뀝니다

- 연산자 = 값을 가지고 계산·비교하는 기호
- 우리가 아는 +, - 부터 처음 보는 //, %, ** 까지
- 3가지 종류로 나뉩니다:

TAKEAWAY

프로그래밍은 문제를 작게 나누고, 실행 순서를 정확히 적는 일입니다.

이론

산술 연산자 한눈에

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

연산자	의미	예	결과
+	더하기	$7 + 3$	10
-	빼기	$7 - 3$	4
*	곱하기	$7 * 3$	21
/	나누기	$7 / 3$	2.33...
//	몫	$7 // 3$	2
%	나머지	$7 \% 3$	1
**	거듭제곱	$2 ** 3$	8

읽는 법

- 표의 첫 행은 기준을 나타냅니다.
- 굵은 색은 오늘 꼭 기억할 부분입니다.

TAKEAWAY

연산자 · 의미 · 예 · 결과

이론

곱하기는 × 가 아니라 *

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 키보드에 × 가 없으니 별표 * 사용
- 나누기도 ÷ 가 아니라 슬래시 /
- 처음엔 어색하지만 곧 익숙해집니다

Python example

```
print(5 * 3) # 15  
print(2 * 2 * 2) # 8
```

TAKEAWAY

키보드에 × 가 없으니 별표 * 사용

이론

나누기 / 는 항상 실수

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- / 결과는 딱 떨어져도 실수(float) → 5.0
- 정수 결과가 필요하다면 다음에 배울 // 사용

Python example

```
print(10 / 2) # 5.0 (4.0이 아니라 5.0!)  
print(9 / 3) # 3.0  
print(7 / 2) # 3.5
```

TAKEAWAY

/ 결과는 딱 떨어져도 실수(float) → 5.0

이론

몫 // — 나눗셈의 정수 부분

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- // = 나눈 몫(정수 부분만)
- 예: 사탕 17개를 5명에게 → 한 명당 3개 (17 // 5)

Python example

```
print(7 // 2) # 3 (7 나누기 2 = 3.5 → 몫 3)
print(10 // 3) # 3
print(17 // 5) # 3
```

TAKEAWAY

// = 나눈 몫(정수 부분만)

이론

나머지 % — 남는 것

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- % = 나눈 후 남는 값
- // 와 % 는 항상 짝꿍: $17 = 5 \times 3 + 2$

Python example

```
print(7 % 2)    # 1  (7 = 2×3 + 1)
print(17 % 5)   # 2  (사탕 17개 5명 나누면 2개 남음)
print(10 % 2)   # 0
```

TAKEAWAY

% = 나눈 후 남는 값

이론

% 의 마법 — 짝수/홀수

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 2로 나눈 나머지가 0이면 짝수, 1이면 홀수
- 응용: 3의 배수? $\rightarrow \% 3 == 0$, 시계(12시간)? $\rightarrow \% 12$

Python example

```
print(10 % 2) # 0 → 나누어떨어짐 → 짝수!  
print(7 % 2) # 1 → 1 남음 → 홀수!
```

TAKEAWAY

2로 나눈 나머지가 0이면 짝수, 1이면 홀수

이론

% 활용 — 시간 변환

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- // 로 큰 단위(분), % 로 남는 작은 단위(초)
- 초 → 시·분·초, 원 → 만원·천원 등에 단골

Python example

```
total = 130 # 130초
minutes = total // 60 # 2분
seconds = total % 60 # 10초
print(f"{minutes}분 {seconds}초") # 2분 10초
```

TAKEAWAY

// 로 큰 단위(분), % 로 남는 작은 단위(초)

이론

거듭제곱 **

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- ** = 거듭제곱 (제곱·세제곱...)
- 넓이(한 변 ** 2), 복리 계산 등에 사용

Python example

```
print(2 ** 3) # 8 (2의 3제곱 = 2×2×2)
print(10 ** 2) # 100 (10의 제곱)
print(5 ** 0) # 1
print(2 ** 10) # 1024
```

TAKEAWAY

**** = 거듭제곱 (제곱·세제곱...)**

이론

연산자 우선순위

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 수학과 동일: 곱·나누기 > 더·빼기
- 헛갈리면 괄호 () 로 묶어 순서를 분명히!
- 괄호는 많이 써도 됩니다 (오히려 안전)

Python example

```
print(2 + 3 * 4) # 14 (곱셈 먼저!)  
print((2 + 3) * 4) # 20 (괄호 먼저)
```

TAKEAWAY

수학과 동일: 곱·나누기 > 더·빼기

이론

복합 대입 연산자

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

줄임	원래 의미
$x += 5$	$x = x + 5$
$x -= 5$	$x = x - 5$
$x *= 2$	$x = x * 2$

읽는 법

- 표의 첫 행은 기준을 나타냅니다.
- 굵은 색은 오늘 꼭 기억할 부분입니다.

TAKEAWAY

줄임 · 원래 의미

비교 연산자

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

연산자	의미	예	결과
==	같다	5 == 5	True
!=	다르다	5 != 3	True
>	크다	5 > 3	True
<	작다	5 < 3	False
=	크거나 같다	5 >= 5	True
<=	작거나 같다	3 <= 5	True

읽는 법

- 비교 결과는 항상 True / False (불리언!)

TAKEAWAY

비교 결과는 항상 True / False (불리언!)

이론

= 와 == 는 다르다

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- = (하나) → 저장(대입)
- == (둘) → 비교(같은지 확인)
- 가장 흔한 실수 TOP3! 조건문에서 = 쓰면 에러

Python example

```
x = 5    # 저장: x에 5를 넣어라  
x == 5   # 비교: x가 5와 같은가? (True)
```

TAKEAWAY

= (하나) → 저장(대입)

이론

문자열도 비교된다

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 글자도 ==, != 로 같은지 비교 가능
- 비밀번호 확인, 메뉴 선택 등에 사용
- 대소문자는 다른 글자로 취급

Python example

```
print("apple" == "apple") # True
print("a" == "A")        # False (대소문자 구분)
print("kim" != "lee")    # True
```

TAKEAWAY

글자도 ==, != 로 같은지 비교 가능

이론

논리 연산자 — 조건 합치기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

연산자	의미	설명
and	그리고	둘 다 참이어야 참
or	또는	하나라도 참이면 참
not	부정	참↔거짓 뒤집기

읽는 법

- 조건이 2개 이상일 때 합쳐서 판단
- 예: "성인 그리고 회원" → 입장 가능

TAKEAWAY

조건이 2개 이상일 때 합쳐서 판단

이론

and / or 자세히

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- and 는 까다롭게(전부), or 는 너그럽게(하나만)

Python example

```
age = 22
# and: 둘 다 참이어야
print(age >= 20 and age < 30) # True (20대인가?)
print(age >= 20 and age < 21) # False

# or: 하나라도 참이면
print(age < 20 or age > 60) # False
print(age < 20 or age == 22) # True
```

TAKEAWAY

and 는 까다롭게(전부), or 는 너그럽게(하나만)

이론

not & 논리 조합

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- not 은 참/거짓을 반대로
- 여러 조건을 괄호로 묶어 복잡한 판단 가능

Python example

```
is_member = True
print(not is_member) # False (뒤집기)

age = 25
print(18 <= age and age <= 64) # True (성인 범위)
print(not (age < 18))         # True (미성년이 아니다)
```

TAKEAWAY

not 은 참/거짓을 반대로

실습

만능 계산기

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
a = int(input("첫 숫자: "))
b = int(input("둘째 숫자: "))

print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} × {b} = {a * b}")
print(f"{a} ÷ {b} = {a / b:.2f}")
print(f"몫={a // b}, 나머지={a % b}")
print(f"{a}가 {b}보다 큰가? {a > b}")
print(f"{a}와 {b}가 같은가? {a == b}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 모든 경우를 확인합니다.

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

생활 속 계산 (택1 이상)

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

따라 하기

- 초 → 시·분·초 변환기 (//, %)
- 피자 나누기: 조각·인원 입력 → 1인당 몇 조각, 남는 조각

2. 실행 결과 확인

3. 값을 바꿔 보기

변형 미션

- 저금 계산기: 목표액·월 저축액 → 몇 개월(//), 마지막 달 부족액(%)
- 나이 → 살아온 날: 나이 × 365

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

휴식

휴식 10분

잠깐 멈추고, 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 조건문 if — 드디어 "상황 따라 다르게" 행동하는 프로그램!

눈 쉬기

질문 정리

파일 저장

다음 실습 준비

TAKEAWAY

쉬는 시간에도 작성한 파일은 저장해 둡니다.

5교시

5교시 — 조건문 if

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분

목표: if/elif/else로 분기하는, 판단하는 프로그램을 만든다.

이론 20분

실습 30분

휴식 10분

이론

조건문이란?

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- "만약 ~라면, 이걸 해라"

예시

- 프로그램이 상황에 따라 다르게 행동하게 만들

주의

- 일상의 조건들:

연결

- 비가 오면 → 우산을 챙긴다

실행하며 확인할 질문

- 점수가 60 이상이면 → 합격
- 잔액이 충분하면 → 결제

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

"만약 ~라면, 이걸 해라"

이론

조건 = 참 또는 거짓

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 조건문의 "조건"은 결국 True / False 로 판가름
- 4교시의 비교·논리 연산자가 여기서 빛을 발합니다
- 조건이 True면 실행, False면 건너뛴

Python example

```
score = 85
print(score >= 60) # True → 실행
print(score >= 90) # False → 건너뛴
```

TAKEAWAY

조건문의 "조건"은 결국 True / False 로 판가름

이론

if 기본 구조

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- if + 조건 + 끝에 콜론 :
- 조건이 참이면 들여쓴 코드 실행

Python example

if 조건:
실행할 코드 # ← 들여쓰기 필수!

```
score = 85  
if score >= 60:  
    print("합격입니다!")
```

TAKEAWAY

if + 조건 + 끝에 콜론 :

이론

콜론(:)을 잊지 마세요

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- if, elif, else 끝엔 항상 콜론
- 빠뜨리면 SyntaxError → 가장 흔한 초보 실수

Python example

```
# 잘못된 예
if score >= 60      # SyntaxError (콜론 없음)
    print("합격")

# 올바른 예
if score >= 60:     # 콜론 있음
    print("합격")
```

TAKEAWAY

if, elif, else 끝엔 항상 콜론

이론

들여쓰기 = 코드의 소속

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 파이썬은 들여쓰기로 코드 묶음을 표시 (다른 언어는 { })
- 같은 칸수로 들여 쓴 줄 = 한 덩어리

Python example

```
if score >= 60:
    print("합격")      # if에 속함 (실행 조건부)
    print("축하합니다") # if에 속함
print("프로그램 끝")  # if와 무관 (항상 실행)
```

TAKEAWAY

파이썬은 들여쓰기로 코드 묶음을 표시 (다른 언어는 { })

이론

들여쓰기 규칙

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 보통 스페이스 4칸 (VS Code에서 Tab 누르면 자동 4칸)
- 한 덩어리는 칸수를 통일
- 들쭉날쭉하면 → IndentationError

Python example

```
if score >= 60:  
    print("합격")  
    print("축하") # 칸수 안 맞음! 에러
```

TAKEAWAY

보통 스페이스 4칸 (VS Code에서 Tab 누르면 자동 4칸)

if - else

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- if 참이면 위쪽, 거짓이면 else 아래 실행
- 둘 중 하나는 반드시 실행됨
- else 에는 조건을 쓰지 않습니다 (콜론만)

Python example

```
age = int(input("나이: "))

if age >= 19:
    print("성인입니다 ")
else:
    print("미성년자입니다 ")
```

TAKEAWAY

if 참이면 위쪽, 거짓이면 else 아래 실행

이론

if - else 흐름 따라가기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 위에서 아래로: 조건 확인 → 참이면 if 블록 → 거짓이면 else 블록
- 둘 다 실행되는 일은 없음 (하나만!)

Python example

```
temperature = 30
if temperature >= 28:
    print("에어컨을 켜세요 ")
else:
    print("창문을 여세요 ")
```

TAKEAWAY

위에서 아래로: 조건 확인 → 참이면 if 블록 → 거짓이면 else 블록

if - elif - else (여러 갈림길)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- elif = "else if" = "그게 아니고 만약 ~라면"
- 갈림길이 3개 이상일 때

Python example

```
score = int(input("점수: "))
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("재수강")
```

TAKEAWAY

elif = "else if" = "그게 아니고 만약 ~라면"

이론

elif는 위에서부터, 하나만!

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 위에서부터 순서대로 검사
- 처음 맞는 것 하나만 실행하고 나머지는 건너뛸

Python example

```
score = 95
if score >= 90: # True! → "A" 출력하고 끝
    print("A")
elif score >= 80: # 검사 안 함
    print("B")
```

TAKEAWAY

위에서부터 순서대로 검사

이론

elif 순서가 중요하다!

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 범위 조건은 좁은 것(큰 수)부터 위에 두어야 함
- 순서가 틀리면 엉뚱한 결과!

Python example

```
# 잘못된 순서
score = 95
if score >= 70:    # 95도 여기서 걸림!
    print("C")    # → "C" 출력 (틀림!)
elif score >= 90:
    print("A")    # 영원히 도달 못 함
```

TAKEAWAY

범위 조건은 좁은 것(큰 수)부터 위에 두어야 함

이론

조건에 and/or 쓰기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- 4교시 논리 연산자를 조건에 결합
- "성인 그리고 돈 충분", "어린이 또는 경로"

Python example

```
age = 25
money = 50000

if age >= 19 and money >= 10000:
    print("입장 + 구매 가능 ")

if age < 8 or age >= 65:
    print("할인 대상 ")
```

TAKEAWAY

4교시 논리 연산자를 조건에 결합

이론

중첩 조건문 (조건 속 조건)

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

개념 포인트

- if 안에 if → 단계별 판단, 들여쓰기 2단계 주의

Python example

```
age = int(input("나이: "))
has_ticket = input("표 있나요?(y/n): ")

if age >= 19:
    if has_ticket == "y":
        print("입장 환영!")
    else:
        print("표를 먼저 구매하세요")
else:
    print("미성년자 입장 불가")
```

TAKEAWAY

if 안에 if → 단계별 판단, 들여쓰기 2단계 주의

이론

흔한 실수 모아보기

개념을 예시로 이해하고, 바로 작은 프로그램으로 확인

핵심

- 콜론 빠뜨림: `if x > 5 → if x > 5:`

예시

- = 와 == 혼동: `if x = 5 → if x == 5`

주의

- 들여쓰기 안 맞음 → `IndentationError`

연결

- `elif` 순서 거꾸로 → 엉뚱한 분기

실행하며 확인할 질문

- `else`에 조건 씌: `else x < 5: → else:`

강의 흐름

- 짧은 정의로 시작
- 일상 예시로 감 잡기
- 코드를 실행해 결과 확인
- 실습 문제에 바로 적용

TAKEAWAY

콜론 빠뜨림: `if x > 5 → if x > 5:`

실습

학점 판별기

바로 해보기: 95 / 85 / 73 / 60 / 45 를 넣어 모든 경우 확인!

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
print("=== 학점 계산기 ===")
score = int(input("점수(0~100): "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(f"{score}점 → {grade} 학점")
```

실습 포인트

- 바로 해보기: 95 / 85 / 73 / 60 / 45 를 넣어 모든 경우 확인!

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

BMI + 등급 (3교시 확장!)

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
weight = float(input("몸무게(kg): "))
height = float(input("키(cm): ")) / 100
bmi = weight / (height * height)

if bmi < 18.5:
    result = "저체중"
elif bmi < 23:
    result = "정상"
elif bmi < 25:
    result = "과체중"
else:
    result = "비만"
print(f"BMI {bmi:.1f} → {result}")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 모든 경우를 확인합니다.

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

윤년 판별기

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
year = int(input("연도: "))
if year % 4 == 0:
    print(f"{year}년은 윤년 ")
else:
    print(f"{year}년은 평년")
```

실습 포인트

- $(year \% 4 == 0 \text{ and } year \% 100 != 0) \text{ or } year \% 400 == 0$
- 2024 / 2025 / 2000 / 1900 넣어 보기
- 도전: 정확한 규칙

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

실습

가위바위보 게임

입력 → 계산/판단 → 출력 흐름을 직접 실행하며 확인

1. 따라 치기

2. 실행 결과 확인

3. 값을 바꿔 보기

실습 코드

```
import random
me = input("가위/바위/보: ")
com = random.choice(["가위", "바위", "보"])
print(f"컴퓨터: {com}")

if me == com:
    print("비겼습니다 ")
elif (me=="가위" and com=="보") or (me=="바위" and com=="가위") or (me=="보" and com=="바위"):
    print("이겼습니다!")
else:
    print("졌습니다 ")
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 모든 경우를 확인합니다.

TAKEAWAY

따라 친 코드를 자기 값으로 바꾸는 순간부터 진짜 실습입니다.

휴식

오프라인 정리 & 휴식

잠깐 쉬고, 온라인에서는 "현실 문제"를 코드로 풀어봅시다!

10 MIN BREAK

설치 · Hello World · 변수/자료형 ·  input/f-string · 연산자 · 조건문

눈 쉬기

질문 정리

파일 저장

다음 실습 준비

TAKEAWAY

쉬는 시간에도 작성한 파일은 저장해 둡니다.